SICP Tutorial

Scheme, Interpreter, Scheme Interpreter

jjppp

NJU SICP

2025 年 12 月 19 日

TOC

1. 课程回顾：Scheme、语法、语义
2. 解释器指北

Scheme: History

> "· · · a **small**, **simple** formalism could serve as the core of a **powerful** and **expressive** programming language."
>
> — Sussman & Steele



图: Gerald Jay Sussman

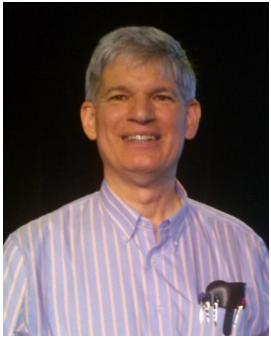


图: Guy L. Steele

1. Gerald Jay Sussman[1] & Guy L. Steele

[1] Also the co-author of SICP in MIT

Scheme: History



图: John McCarthy[1]

1. Gerald Jay Sussman[2] & Guy L. Steele
2. Lisp 的一个方言

[1] 1971 Turing Award for Artificial Intelligence and Programming Languages
[2] Also the co-author of SICP in MIT

Scheme: History



图: IBM 704

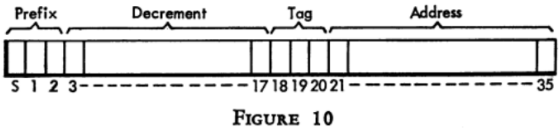


FIGURE 10

图: 数据在 IBM 704 计算机中的存储[1]

1. Gerald Jay Sussman[2] & Guy L. Steele
2. Lisp 的一个方言
3. 1970s，继承了奇怪的“术语”：CAR、CDR
   源自当年的硬件设计：**C**ontents of **A**ddress/**D**ecrement part of the **R**egister

[1] Computer History: https://archive.computerhistory.org
[2] Also the co-author of SICP in MIT

Scheme 设计哲学：极简

在开电脑都费劲的年代，写程序并非易事

- 先 `#include` 10 个头文件？[a]
- 定义 95 个关键字？[b]
- 写 2104 页 ISO 标准？[c]

[a] C++
[b] C++
[c] 还是 C++，ISO/IEC 14882:2024



图: 课程主页获取和 Scheme 有关的一切

Scheme 设计哲学：极简

在开电脑都费劲的年代，写程序并非易事

- 先 `#include` 10 个头文件？[a]
- 定义 95 个关键字？[b]
- 写 2104 页 ISO 标准？[c]

[a] C++
[b] C++
[c] 还是 C++，ISO/IEC 14882:2024



图: 课程主页获取和 Scheme 有关的一切

极简、极简、再极简

- 基本数据（够用）：2025, 3.14, -1, #t
- 复杂数据都是 List：`(list 2 0 2 5)`
- 程序本身也是 List：`(+ 2 (* 3 4))` ⇒ `(list '+ 2 (list '* 3 4))`

Scheme 语义：Call Expression

对 <operand1> <operand2> ... 执行<operator> 操作

(<operator> <operand1> <operand2> ...)

求值顺序

1. 先对<operator> 求值，得到一个procedure[a]
2. 再按照顺序对 <operand1> 求值、对 <operand2> 求值...
3. 把 <operand1> ... 的求值结果交给procedure

[a] Just another name for function

省流：一般情况和 Python 的规则一样

Scheme 语义：Special Forms

当operator 是一些特殊操作时，求值规则发生变化

(if <condition> <if-true> <if-false>)

求值顺序：以 `if` 为例

1. 先对<condition> 求值
2. 如果为真，那么对 <if-true> 求值
3. 否则对 <if-false> 求值

省流：特殊情况特殊处理

TOC

1 课程回顾：Scheme、语法、语义

2 解释器指北

Interpreter：翻译官



“我想用上计算机”

2 ** 5

2 ** 5 是 32

python

“包在我身上！”

帮我算 2 的 5 次方

2 的 5 次方是 32

Interpreter：翻译官



“我想用上计算机”

(expt 2 5)

“包在我身上！”

2 ** 5



“包在我身上！”

帮我算 2 的 5 次方

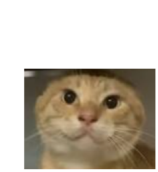


2 的 5 次方是 32

2 ** 5 是 32

(expt 2 5) 是 32

Interpreter：翻译官



“我想用上计算机”

(expt 2 5)

(expt 2 5) 是 32

“包在我身上！”

2 ** 5

2 ** 5 是 32

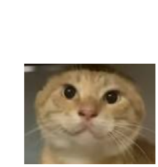
“包在我身上！”

帮我算 2 的 5 次方

2 的 5 次方是 32

- 使用抽象：耄耋不需要知道 Scheme 解释器的细节，不关心谁实现 Scheme

Interpreter：翻译官



“我想用上计算机”

(expt 2 5)

“包在我身上！”

2 ** 5

“包在我身上！”

帮我算 2 的 5 次方

2 的 5 次方是 32

2 ** 5 是 32

(expt 2 5) 是 32

1. 使用抽象：耄耋不需要知道 Scheme 解释器的细节，不关心谁实现 Scheme
2. 实现抽象：程序员（we guys）需要知道，并正确实现这个过程

Interpreter：架构

解释器一般怎么用？

1. 输入代码（`print 'SICP 'love`）
2. 解释器对代码求值
3. 解释器告诉我们结果 (SICP love)
4. 重复步骤 1-3

对应于四个步骤

1. Read
2. Evaluate
3. Print
4. Loop

几乎所有解释器（在原理上）都长这样：

```
while has_user_input():
    code = read_user_input()
    result = evaluate(code)
    print(result)
```

proj04 也不例外

Implementer vs Implemented

核心问题：用谁实现谁？

- 用 Python 实现 Scheme (Proj04)
- 用 C 实现 Python (CPython)
- 用 Scheme 实现 Scheme （Meta-Circular Interpreter）
- ...

Implementer vs Implemented

核心问题：用谁实现谁？

- 用 Python 实现 Scheme (Proj04)
- 用 C 实现 Python (CPython)
- 用 Scheme 实现 Scheme （Meta-Circular Interpreter）
- ...

还能套娃

1. 用 Python 写个 Scheme 解释器
2. 再用 (跑在 Python 写的解释器上的 Scheme) 写个 Python 解释器
3. 再用 (跑在 (用跑在 Python 写的解释器上的 Scheme) 写的解释器上的 Python) 写个 Scheme 解释器
4. ...

Scheme 在 Python 中的表示

Scheme			Python	
Number	`2025`	$\Longleftrightarrow$	`2025`	`int/float`
Symbol	`print`	$\Longleftrightarrow$	`'print'`	`str`
Pair	`'(114 . 514)`	$\Longleftrightarrow$	`Pair(114, 514)`	`Pair`
List	`'(114 514)`	$\Longleftrightarrow$	`Pair(114, Pair(514, nil))`	`Pair/nil`
代码	`(print 114514)`	$\Longleftrightarrow$	`Pair('print, Pair(114514, nil))`	`Pair/nil`

关于 proj04 的小 Tips:

1. Scheme list $\neq$ Python list
2. Scheme list $\neq$ Scheme Pair (why?)
3. 函数调用的参数是一个 Scheme list
4. 注意 `nil`

Proj04 生存指南 I

DDL: 2025-12-26 16:00

Do the right thing, in the smart way

1. proj04 非常复杂 ⇒ “瞪眼法” 很难再适用
2. 用聪明的办法理解程序的行为（`print`、Debug）

阅读，阅读，阅读

- 阅读 handout
- 阅读相关的函数实现，对 proj 有整体理解
- 阅读 Scheme 的 manual

尽快开始

不要拖延

已提交人数（仅含学生）

68 人 查看列表

Proj04 生存指南 II—Debugger

待会儿要用到的妙妙工具

```
{
    "name": "Debugging Scheme Interpreter",
    "type": "debugpy",
    "request": "launch",
    "console": "integratedTerminal",
    "justMyCode": true,
    "program": "scheme.py",
    "args": [
        "-i",
        "tests.scm",
    ]
}
```

从具体问题入手

prob07 中 `do_lambda_form` 的参数 `expressions` 到底是什么？

Take-aways

1. 一切 “怪异” 皆有其原因（`car cdr`）
2. Scheme 是一门简洁、漂亮的语言（对比 C++）
3. 解释器的原理、4 个步骤（REPL）
4. Do the right thing, in the smart way（Debugger）